

Energy-Efficient Deep Learning Without Backpropagation: A Rigorous Hardware-Validated Benchmarking Study of Forward-Only Algorithms

Przemysław Spyra^[0009–0009–3001–2604] and Witold Dzwiniel^[0000–0001–8321–5928]

AGH University of Krakow, Faculty of Computer Science, Kraków, Poland
pspyra@agh.edu.pl, dzwiniel@agh.edu.pl

Abstract. Backpropagation (BP) powers modern deep learning, yet it is biologically implausible and computationally awkward: every activation must stay in memory until a global backward sweep consumes it, and that sweep locks each layer until the one above has finished. Forward-only algorithms, from Hinton’s Forward-Forward (FF) through Cascaded Forward (CaFo) to Mono-Forward (MF), replace the global gradient with local learning signals and are promoted as more efficient on the basis of FLOP counts rather than measurements. We test that claim on hardware. Across almost 1000 runs on four vision benchmarks (MNIST, Fashion-MNIST, CIFAR-10 and CIFAR-100), we read energy, power and memory from an NVIDIA A100 GPU and compared each algorithm against a tuned BP baseline. Removing the backward pass alone saves nothing: FF costs $4.1 \times$ to $6.3 \times$ BP’s energy and CaFo $3.0 \times$ to $7.1 \times$. Mono-Forward is different. Given the same number of data passes as BP, it uses 12.5 % to 19.9 % less energy and up to 27.5 % less memory on the 3×2000 multilayer perceptrons, the largest configurations tested, at lower power in comparable time; on CIFAR-100 its test accuracy is statistically equivalent to BP’s. A pre-registered ablation identifies gradient detachment, not MF’s auxiliary losses, as the dominant factor tested. Trained to convergence, MF exceeds BP’s test accuracy on CIFAR-10 and CIFAR-100 by 3.90 and 5.04 percentage points, with $3.5 \times$ to $4.5 \times$ as many passes. A forward-only algorithm can thus match backpropagation’s accuracy at lower energy and, given more passes, surpass it; faster convergence is the remaining target for an end-to-end gain.

Keywords: Backpropagation-free learning · Energy efficiency · Hardware benchmarking · Forward-only algorithms · Deep neural networks

1 Introduction

The energy cost of training deep neural network (DNN) models is no longer negligible. Training footprints have grown faster than the efficiency of the underlying hardware [26, 18], and the gap between what accelerators deliver and what workloads actually draw is now a first-order engineering concern [8, 28].

What the training algorithm itself costs is therefore worth measuring directly. Backpropagation (BP) [21] sits at the centre of that cost. It requires every intermediate activation of the forward pass to remain live until the backward pass consumes it, which fixes a memory floor proportional to depth and width, and it serialises the backward sweep, which fixes a dependency chain that no amount of parallel hardware removes [7, 13].

Forward-only algorithms remove the backward sweep. Forward-Forward (FF) [5], Cascaded Forward (CaFo) [29] and Mono-Forward (MF) [3] each replace the global gradient with signals computed locally, and each is proposed, at least in part, on efficiency grounds. The evidence offered for that efficiency is usually analytical: a count of floating-point operations (FLOPs), a parameter count, an argument about what need not be stored. Very little of it is measured on hardware, and an analytical proxy cannot see the two things that dominate a real training run, which are how much power the device draws while it works and how many passes over the data the algorithm needs before it stops.

This paper measures both. We instrument every run through the NVIDIA Management Library (NVML), the driver’s own telemetry interface, and read power, utilisation, clock and memory from the device at 5 Hz while the workload executes. Across about 1000 runs on four vision benchmarks (MNIST, Fashion-MNIST, CIFAR-10 and CIFAR-100), and given the same number of data passes as BP, Mono-Forward uses 12.5 % to 19.9 % less energy and 24.6 % to 27.5 % less allocator memory than backpropagation on the multilayer perceptrons (MLPs) with three hidden layers of 2000 units (3×2000) on CIFAR-10 and CIFAR-100, the largest configurations tested; on the 2×1000 MLPs trained on MNIST and Fashion-MNIST the energy saving is 2.2 % to 4.3 %. MF is cheaper because it draws less power, not because it runs faster, so the comparison does not depend on when a run was allowed to stop.

Under a fixed budget the cheaper pass appears as lower cost. Trained to its own convergence, MF reaches test accuracy above BP’s by 3.90 and 5.04 percentage points (pp) on CIFAR-10 and CIFAR-100, at the price of more passes.

This paper makes four contributions. The first is the matched data-pass protocol, a comparison in which MF receives at least as many data passes as BP actually used, so the stopping rule cannot confound the result; under it MF uses 12.5 % to 19.9 % less energy and 24.6 % to 27.5 % less allocator memory on the 3×2000 MLPs (Section 5.1, Table 2). The second is that the advantage is a *power* advantage at near-equal wall-clock time and that it is larger in the larger configurations tested, from no memory advantage at 1.8 M weights to 27.5 % at 14.2 M (Figure 1). The third is a pre-registered ablation ladder that identifies gradient detachment, rather than the localised objectives, as the dominant factor among those tested: a deep supervision control is *worse* than plain BP on both CIFAR tasks, while MF exceeds it by 5.56 and 6.10 pp in test accuracy (Section 5.3). The fourth observation is that the convergence rate is the only remaining barrier to MF achieving an end-to-end energy advantage. This rate, quantified as a $3.5 \times$ – $4.5 \times$ pass multiplier, defines the target that schedule design should aim to meet (Section 6).

2 Background and Related Work

The cost of the backward pass. BP computes exact gradients by storing the forward activations and traversing the graph in reverse. The consequences are structural: activation memory scales with depth, width and batch size, and the backward traversal cannot begin until the forward pass ends (forward locking) or be split across layers without approximation (backward locking) [7]. Alternatives include predictive coding [19, 22], Hebbian and associative schemes [9, 23, 24], and zeroth-order methods for large-model fine-tuning [16]; Lillicrap et al. [13] survey the space that the biological case against BP opens: the symmetric feedback weights and separate backward phase that BP requires are difficult to reconcile with what is known about cortical circuits.

Forward-only and local-loss methods. We write vectors in bold lower case and matrices in bold upper case. A network with hidden layers $i = 1, \dots, L$ has a weight matrix $\mathbf{W}_i$ per layer and activation vectors $\mathbf{a}_i = f(\mathbf{W}_i \mathbf{a}_{i-1})$, where $\mathbf{a}_0 = \mathbf{x}$ is the input, f is the activation function and biases are omitted. Forward-Forward (FF) [5] trains each layer to separate positive (real) from negative (corrupted) data by a goodness score, the sum of its squared activations, so no gradient crosses a layer boundary. Cascaded Forward (CaFo) [29] attaches a predictor to every block of a convolutional cascade and trains each predictor independently of the others; a variant first pre-trains the blocks with direct feedback alignment (DFA). Mono-Forward (MF) [3] attaches to each activation $\mathbf{a}_i$ a learned projection matrix $\mathbf{M}_i$ that maps it to class logits $\mathbf{M}_i \mathbf{a}_i$, trains each layer on a local cross-entropy loss over those logits, and trains the layers one after another. Related local-objective work has scaled layer-wise training to larger settings [20] and analysed it in the infinite-width limit [6], where greedy layer-wise optimisation reduces the effective complexity of the learned representation.

Deep supervision. Auxiliary classifiers attached to intermediate layers predate the forward-only literature [12] and are the obvious control for it: they supply the same local supervisory signal while leaving the global gradient intact, and layer-wise objectives have been studied as a training device in their own right [14]. Section 5.3 applies that control.

Hardware and energy. Analytical proxies systematically misestimate real consumption [27, 2], because they cannot see clock behaviour, utilisation or the fixed costs of a kernel launch. Direct device telemetry is the alternative, and recent forward-only work has begun to adopt it [17]. We use the NVIDIA Management Library (NVML) throughout and name the instrument behind every quantity.

3 Mono-Forward and the Cost Hypothesis

MF trains stage i of an MLP by minimising the cross-entropy of $\mathbf{M}_i \mathbf{a}_i$ against the true class label, so that each layer learns to classify the input on its own. Gradients of this loss are computed with respect to $\mathbf{W}_i$ and $\mathbf{M}_i$ only: the input $\mathbf{a}_{i-1}$ is treated as a constant, detached from the computational graph, so no gradient propagates to earlier layers. Training proceeds one stage at a time, so a network with L hidden layers has $L + 1$ stages, one per projection including $\mathbf{M}_0$ on the raw input. At inference the network is a standard feedforward stack with $\mathbf{M}_L$ as the readout.

Three consequences follow, and they are the reason to expect a cost advantage. No activation needs to outlive its stage, so the allocator high-water mark is set by the widest single layer rather than by the whole forward pass. Each optimiser step updates one weight matrix and one projection, so the arithmetic per step is smaller and the kernels narrower. Finally, every layer below the current stage is frozen throughout that stage, so its activations depend only on the sample and can be cached, which no jointly trained network permits. The countervailing effect is equally structural: stage i must reconstruct $\mathbf{a}_{i-1}$ before it can train, which yields an $O(L^2)$ total when recomputation is used. Our experiments recompute, and Section 5.5 evaluates whether caching is beneficial. MF spends less work per step and more per epoch, so the sign of the net effect is an empirical question about a device, and the rest of this paper answers it.

4 Measuring Cost Fairly: Two Protocols

Energy and time are integrals over the duration of training, so whatever decides when training stops also shapes the efficiency comparison. Every result below uses one stopping rule, applied identically to every algorithm, and we report two protocols rather than one, because no single protocol answers both of the questions a practitioner asks: what an algorithm costs at its best, and what it costs for the same number of data passes.

Protocol A: own convergence. Every algorithm stops on validation loss with the same minimum improvement (0), the same patience (20) and the same cap, an upper bound of 100 epochs on each training unit (the whole network for BP, each stage for MF). Each method therefore trains until it stops improving. This answers “what does it cost to get the best this method can do”, and it is the protocol under which MF’s accuracy advantage appears.

Protocol B: matched data-pass protocol. MF is given the number of data passes BP actually spent, and nothing else changes. The budget is

$$E_{\text{MF}}^{\text{per stage}} = \left\lceil \frac{\bar{E}_{\text{BP}}}{S} \right\rceil, \quad E_{\text{MF}}^{\text{total}} = S \cdot E_{\text{MF}}^{\text{per stage}}, \quad (1)$$

where $\bar{E}_{\text{BP}}$ is BP’s mean epoch count under Protocol A and S is MF’s stage count.

Table 1. The matched data-pass budget of Equation 1, with BP’s realised mean epoch count measured under Protocol A.

Configuration	BP mean epochs	MF stages	Per stage	MF total
MNIST 2×1000	25.4	3	9	27
Fashion 2×1000	25.6	3	9	27
CIFAR-10 3×2000	52.0	4	13	52
CIFAR-100 3×2000	42.9	4	11	44

Rounding up with the ceiling function $\lceil \cdot \rceil$, which is unrelated to the epoch cap of Protocol A, is deliberate: it can only give MF *more* passes than BP had, so a saving measured under this protocol cannot be an artefact of a smaller budget. Table 1 gives the resulting epoch budgets. BP itself is not rerun under Protocol B: it keeps its own early stopping and is never pushed past the point it chose to stop, which is why the BP column is the same in both protocols. The budget equalises data passes, not optimiser work per layer: because MF trains one stage at a time, each MF layer receives $\lceil \bar{E}_{\text{BP}}/S \rceil$ updates where each BP layer receives $\bar{E}_{\text{BP}}$, which is part of why the budget costs MF accuracy on CIFAR-10.

Early stopping stays *enabled* with patience equal to the cap, so it cannot fire inside the budget, and MF still runs the per-epoch validation pass that BP runs.

What else is held fixed. Each algorithm is run on the architecture specified in its source publication. For the MLP experiments, CIFAR inputs are flattened following the MF protocol [3]. Each BP baseline uses the same module list, with only the algorithm-specific components removed. Optuna [1] tuned every algorithm and baseline under an identical trial budget and epoch cap, unpruned, with the sampler seed derived from the study name; BP and MF search two hyperparameters (learning rate and weight decay), the ladder controls BP-DS and MF-Joint of Section 5.3 three (adding the auxiliary-loss weight), all at 50 trials. All MLPs use ReLU and a batch size of 128, with Kaiming uniform initialisation [4] for every weight and projection; BP and BP-DS use AdamW [15], MF-Joint and MF stages use Adam [10] on a local cross-entropy. All algorithms share one PyTorch codebase and the same `nn.Linear` primitives, differing only in the loss and in where `detach()` is called, so no method gains a fused kernel the others lack.

Instruments. A monitor thread running alongside each job queries the NVIDIA Management Library (NVML) at 5 Hz for power (`nvmlDeviceGetPowerUsage`), utilisation (`nvmlDeviceGetUtilizationRates`), streaming multiprocessor clock (`nvmlDeviceGetClockInfo`) and device memory (`nvmlDeviceGetMemoryInfo`); energy in Wh is the trapezoidal integral of power over the run. Wall-clock time comes from the monotonic clock of the host. The NVML memory reading covers the whole device and includes a fixed ≈ 870 MiB CUDA context that an algorithm can neither reduce nor control. Every memory number in this paper is instead the per-process allocator high-water mark, `torch.cuda.max_memory_allocated`.

Setup and statistics. Runs used NVIDIA A100-SXM4-40GB graphics processing units (GPUs) on AMD EPYC 7742 nodes of the Athena cluster at ACK Cyfronet AGH, driver 595.71.05, Python 3.10.4, PyTorch 2.4.0+cu121. CIFAR runs used random horizontal flips and 4-pixel-padded random crops, applied identically to every algorithm and its baseline. MNIST and Fashion-MNIST were not augmented. Accuracy is measured on the held-out test split, and results are mean $\pm$ standard deviation (SD) over the seeds stated per table. *The ablation ladder of Section 5.3 is pre-registered:* the analysis script fixing the paired-by-seed primary test, the Holm-Bonferroni correction within each metric family, the 10 000-resample bootstrap intervals and the ± 0.25 pp equivalence margin was written and committed before any ladder result was inspected. Parity claims require the two one-sided tests (TOST) procedure [25, 11], which declares two arms equivalent only when the 90% confidence interval (CI) of their difference lies inside the margin. A contrast is EQUIVALENT when TOST succeeds at $\alpha = 0.05$, otherwise DIFFERENT when the difference test rejects, and otherwise INCONCLUSIVE. The matched data-pass protocol of Section 5.1 was *not* pre-registered; it is a diagnostic, analysed with the same machinery and margins, not a confirmatory test.

5 Results

5.1 The Matched Data-Pass Protocol: MF Is the Cheaper Algorithm

Table 2 summarises the main experimental results. Given the budget of Equation 1, MF draws 19.9% less GPU energy than BP on the CIFAR-10 3×2000 MLP and 12.5% less on CIFAR-100, holds 27.5% and 24.6% less per-process allocator memory, and runs at 63.3 W and 62.8 W of mean board power against BP’s 76.8 W and 72.0 W. All of these contrasts are DIFFERENT at $n = 20$ paired seeds, and on CIFAR-100 accuracy is statistically equivalent to BP’s. Three properties of these measurements make the result stronger than the percentages suggest.

The memory figures are exact. Peak allocator occupancy has zero seed-to-seed standard deviation in every arm of every configuration, because the allocation schedule of a fixed architecture at a fixed batch size is deterministic, so the 82.2 MiB and 74.4 MiB that MF saves need no confidence interval.

The energy saving is a power saving. Wall-clock time under the matched data-pass protocol is not reduced: in the column order of Table 2 the four contrasts give +3.8, -0.8 , -2.8 and $+0.4$ %, of which CIFAR-100 is EQUIVALENT within a ± 5 % margin, MNIST and CIFAR-10 are INCONCLUSIVE, and Fashion-MNIST is DIFFERENT with MF the faster. Power is a rate, so no stopping rule can influence it, though at this utilisation it is specific to this device (Section 6).

On the most demanding task the saving costs no accuracy. On CIFAR-100, with 100 classes, MF reaches accuracy statistically equivalent to BP’s under the budget ($+0.02$ pp, 90% CI $[-0.13, 0.16]$, TOST EQUIVALENT) while spending 12.5% less energy and 24.6% less memory. Elsewhere the budget costs MF some accuracy, 1.11 pp on CIFAR-10 $[-1.32, -0.91]$ and 0.22 and 0.49 pp on the two 2×1000 tasks, as expected of a method stopped short of convergence (Section 5.4); the energy saving points the same way on all four and is resolved on three.

Table 2. Matched data-pass protocol (Protocol B). **MF** receives the budget of Table 1; **BP** is its own Protocol A run. Energy is the trapezoidal integral of the 5 Hz NVML power trace; memory is `torch.cuda.max_memory_allocated`, per process; power is the trace mean. Δ rows are MF relative to BP, so negative is cheaper. MF arms have $n = 20$; BP arms have $n = 60, 122, 20, 20$ by column. Memory has zero seed-to-seed SD.

	MNIST 2×1000	Fashion 2×1000	CIFAR-10 3×2000	CIFAR-100 3×2000
MF epoch budget	27	27	52	44
<i>Test accuracy (%)</i>				
BP	98.20	89.41	62.47	32.90
MF	97.98	88.92	61.36	32.92
Δ (pp)	−0.22	−0.49	−1.11	+0.02
<i>GPU energy (Wh)</i>				
BP	1.079	1.100	5.393	4.141
MF	1.055	1.053	4.320	3.624
Δ (%)	−2.2	−4.3	−19.9	−12.5
<i>Peak allocated memory (MiB, per process)</i>				
BP	50.9	50.9	299.0	302.5
MF	51.0	51.0	216.8	228.1
Δ (%)	+0.3	+0.3	−27.5	−24.6
<i>Mean board power (W)</i>				
BP	62.9	61.5	76.8	72.0
MF	59.3	59.3	63.3	62.8
Δ (%)	−5.8	−3.6	−17.6	−12.8
<i>Wall-clock time Δ (%)</i>	+3.8	−0.8	−2.8	+0.4

5.2 The Advantage Is Larger in the Larger Configurations

The four configurations span a nearly eightfold range of trainable weights, from 1.79 M on the 2×1000 MLPs to 14.16 and 14.34 M on the 3×2000 MLPs, and every cost axis moves the same way across that range (Figure 1). Energy saving rises from 2.2 % and 4.3 % on the smaller configurations to 12.5 % and 19.9 % on the larger; power saving from 3.6 % and 5.8 % to 12.8 % and 17.6 %; and memory from a 0.3 % *deficit*, which is the 0.14 MiB of extra projection parameters, to a saving of 24.6 % and 27.5 %.

The mechanism is visible in the utilisation trace. Over these four configurations **BP** holds the device at 11.5 % mean utilisation and never exceeds 17.3 %, while **MF** averages 6.0 % and never exceeds 8.0 %. Both are far from saturation: the device idles between kernels for most of each run, and **MF** keeps it busy about half as long. We read the widening gap as an effect of the narrower per-stage kernels, which the trace is consistent with but does not prove.

The four configurations give only two width levels, and width moves together with depth and task across them, so Figure 1 cannot separate width from scale. It does show that the advantage is not a small-model artefact: **MF** does better on the larger networks.

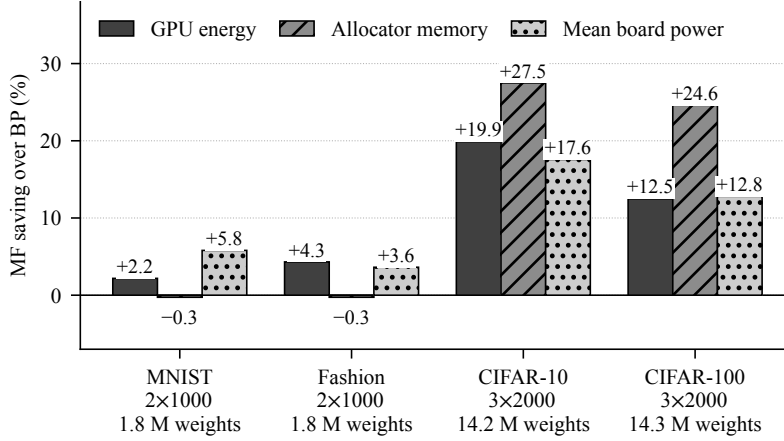


Fig. 1. Cost saving of MF over BP under the matched data-pass protocol, ordered by trainable weights. Positive is cheaper. Every axis is larger at the 3×2000 size, though not monotone within a size; the memory advantage only appears at the larger size, where the 0.14 MiB of extra projection parameters stops dominating. Bars are seed means, $n = 20$ for MF and 20 to 122 for BP. Memory is deterministic within an arm.

5.3 Where the Accuracy Comes From

MF differs from BP in three ways at once: it adds auxiliary losses, it adds the projections $\mathbf{M}_i$, and it detaches the gradient. We therefore interpolate between BP and MF in single structural steps and measure each (Table 3, Figure 2):

Rung	Readout	Auxiliary losses	Gradients	Optimiser
BP	output layer	no	global	AdamW
BP-DS	output layer	yes	global	AdamW
MF-Joint	$\mathbf{M}_L$	yes	global	Adam
MF	$\mathbf{M}_L$	yes	<i>local</i>	Adam

Each rung is tuned independently, so every step also moves the learning rate and weight decay, and BP-DS→MF-Joint crosses AdamW to Adam: that step is not attributable to the readout alone. The step the mechanism rests on, MF-Joint→MF, holds the optimiser family fixed. BP-DS is the standard deep supervision control: MF’s auxiliary classifiers attached to BP with nothing detached.

On the two CIFAR 3×2000 MLPs, deep supervision on its own is *worse* than plain BP, by 1.65 pp $[-1.85, -1.45]$ on CIFAR-10 and 1.06 pp $[-1.26, -0.87]$ on CIFAR-100. MF beats that control (Table 3) by 5.56 pp $[5.37, 5.75]$ and 6.10 pp $[5.88, 6.32]$, these two contrasts at Holm-corrected $p < 10^{-20}$ and the two above at $p < 10^{-8}$; all are 95% bootstrap intervals over 20 paired seeds. Auxiliary supervision therefore does not explain the accuracy; added to BP, it lowers BP’s accuracy. Among the factors tested, detaching the gradient is the largest single contributor: the MF-Joint→MF step, which detaches it with the optimiser family held fixed, moves accuracy by +4.10 and +3.18 pp on CIFAR-10 and CIFAR-100.

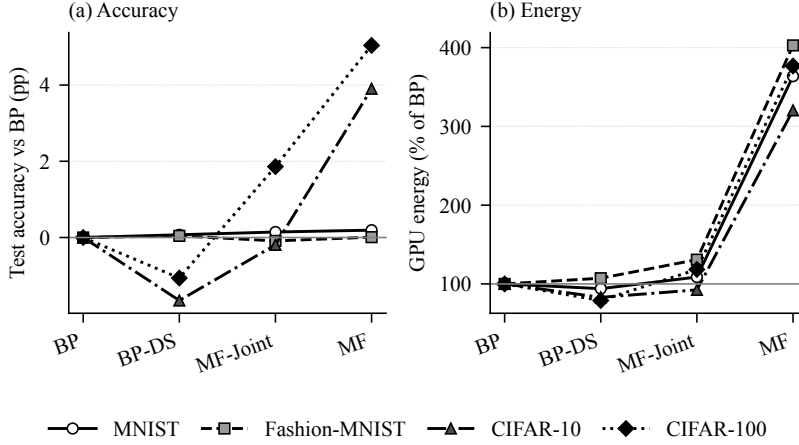


Fig. 2. The ablation ladder, one structural change per step. (a) Test accuracy against BP. Deep supervision (BP-DS) falls below BP on both CIFAR tasks; the readout step recovers most or all of that, and the detach is the largest single gain on both. (b) GPU energy as a percentage of BP. The same step carries essentially the whole cost increase. Markers are seed means over 20 to 122 seeds per rung.

On the two 2×1000 MLPs the same comparison is a tie: MF leads BP-DS by 0.11 pp $[-0.01, 0.23]$ on MNIST and trails by 0.06 pp $[-0.17, 0.06]$ on Fashion-MNIST, both EQUIVALENT within the pre-registered ± 0.25 pp equivalence margin. We report both halves of the answer: on the small tasks the experiment does not distinguish an MF-specific benefit from auxiliary supervision, and on the harder ones auxiliary supervision lowers accuracy and the detach is the largest single step.

In energy, the detach step is also the expensive one: it adds 228 to 272 pp of BP’s energy across the four configurations, while auxiliary supervision and the readout together move energy by at most 31 pp, and on CIFAR-10 move it in the opposite direction (-7.5 pp). The detach step is thus the largest contributor to both the accuracy gain and the cost, which is why the two protocols disagree and why neither alone is a complete answer.

5.4 Own Convergence: Accuracy BP Does Not Reach

The matched budget of Section 5.1 stops MF early, and lifting it shows what the method is actually capable of. Allowed instead to train until it stops improving, MF *surpasses* BP by 3.90 pp $[3.68, 4.11]$ on CIFAR-10 and 5.04 pp $[4.81, 5.24]$ on CIFAR-100, both pre-registered 95% bootstrap intervals over 20 paired seeds. On the two 2×1000 tasks it matches BP on Fashion-MNIST and leads it on MNIST, where the pre-registered contrast is DIFFERENT at $+0.25$ pp $[0.13, 0.38]$ (Table 4). Under the matched data-pass protocol MF’s energy is 98 %, 96 %, 80 % and 88 % of BP’s (Table 2); at its own convergence, 320 % to 403 % (Table 4). Even at its own convergence MF draws 19.1 % and 14.3 % less board power than BP on CIFAR-10 and CIFAR-100; the extra cost is time, not power.

Table 3. The BP→MF ablation ladder under Protocol A, one structural change per step; rungs are tuned independently, so BP-DS→MF-Joint also crosses AdamW to Adam. Δ is accuracy against BP in pp, as a difference of arm means; where arms have unequal n the paired contrast differs (MNIST, +0.25 pp). E is GPU energy as a percentage of BP’s. Seeds per rung: 20 to 122. This analysis is pre-registered; intervals and Holm-corrected p -values for every contrast are in the released `ladder_analysis.json`.

	MNIST 2×1000	Fashion 2×1000	CIFAR-10 3×2000	CIFAR-100 3×2000
<i>Test accuracy against BP (pp)</i>				
BP	+0.00	+0.00	+0.00	+0.00
BP-DS	+0.07	+0.04	−1.65	−1.06
MF-Joint	+0.14	−0.09	−0.19	+1.86
MF	+0.19	+0.01	+3.90	+5.04
<i>GPU energy (% of BP)</i>				
BP	100	100	100	100
BP-DS	94	107	83	79
MF-Joint	109	131	92	118
MF	364	403	320	377

The two protocols do not contradict each other. Per data pass MF is the cheaper algorithm, by the margins of Table 2. It then takes $3.5\times$ to $4.5\times$ as many passes to reach its own stopping point, from the realised Protocol A epoch medians (90 to 207 MF epochs against BP’s 26 to 52), and the product is $3.2\times$ to $4.0\times$ BP’s energy end to end. Wall-clock time grows by a similar factor, $3.6\times$ to $4.4\times$, consistent with the near-equal time per pass measured under Protocol B. The per-pass advantage remains; the additional passes outweigh it, and Section 6 treats the multiplier as the open problem.

5.5 Quantifying Two Objections

Is the recomputation simply an unoptimised implementation? MF recomputes the frozen prefix at every stage, an $O(L^2)$ cost that caching should remove, so we implemented the cache. It is bit-exact on the two 2×1000 MLPs: over 1200 and 3600 optimiser steps the loss and weights match the recomputing path exactly. It saved no time: on-device caching moved wall-clock by +3.8% and +2.7% and cost 374% and 404% more per-process memory; host caching was worse on every axis. The cache does not help, because at these widths the device is launch-bound rather than arithmetic-bound. Only the local method can cache at all: in BP, BP-DS and MF-Joint a joint gradient makes a cached activation stale within one optimiser step.

Does BP benefit from more mature libraries? If kernel maturity decided the comparison, BP would run near saturation, unlike the forward-only methods. Utilisation traces show otherwise: on the four ladder configurations no method exceeds 20.3% mean utilisation, and across the entire study, the wider FF MLPs included, no run exceeds 38.0% mean utilisation, still far from saturation.

Table 4. Own convergence (Protocol A). Ratios are MF over BP. Δ pp is a difference of arm means; on MNIST, where n differs between arms, the paired contrast is +0.25 pp. Seeds: $n = 20$ per arm on the CIFAR configurations, $n = 60/20$ and $n = 122/48$ on MNIST and Fashion-MNIST.

Configuration	BP acc. (%)	MF acc. (%)	Δ (pp)	Time $\times$	Energy $\times$
MNIST 2×1000	98.20	98.40	+0.19	3.64	3.64
Fashion 2×1000	89.41	89.42	+0.01	4.21	4.03
CIFAR-10 3×2000	62.47	66.37	+3.90	3.96	3.20
CIFAR-100 3×2000	32.90	37.94	+5.04	4.39	3.77

Every arm is launch-bound, so BP does not win by saturating the device with more efficient arithmetic. This does not entirely exclude an advantage from implementation maturity, but in these measurements the end-to-end difference between MF and BP lies in the number of passes, not in the cost of a pass.

5.6 The Other Two Algorithms

FF and CaFo are the rest of the forward-only landscape, and neither reproduces MF’s combination of a per-pass cost advantage with competitive accuracy. FF trails BP in accuracy on its native MLPs by 0.01 to 1.10 pp across its four arms, and requires $4.1 \times$ to $7.4 \times$ BP’s wall-clock time and $4.1 \times$ to $6.3 \times$ its energy, with memory at $0.91 \times$ to $1.06 \times$ BP’s allocator peak. Removing the backward pass therefore does not by itself produce the saving. CaFo costs $3.0 \times$ to $7.1 \times$ BP’s energy. It lands within 0.4 pp of BP on the two MNIST-scale convolutional neural networks (CNNs) but loses 18.8 and 16.9 pp on CIFAR-10 and CIFAR-100; its direct-feedback-alignment variant narrows that to 7.1 and 8.5 pp at a further $1.5 \times$ to $2.6 \times$ the energy. Forward-only training is not generically cheaper: the advantage measured in this paper belongs to gradient locality as MF implements it, not to the family. Seeds are five per arm for the CNNs and seven per arm for the MLPs.

6 Discussion and Limitations

Why gradient detachment might improve accuracy. The ladder shows a mechanism rather than just a performance gap: auxiliary supervision alone makes BP worse on the CIFAR tasks by 1.06–1.65 pp, precisely where MF gains 3.90–5.04 pp. This means the improvement comes from detaching the gradient, not from the local objective. One interpretation is that greedy layer-wise optimisation constrains each layer’s effective capacity and so generalises better than joint optimisation [20, 6]; four configurations and one architecture family cannot establish this, so we state it as a hypothesis.

The open problem: convergence rate, not per-unit cost. Among the factors this study measures, one stands between MF and an end-to-end energy advantage.

MF needs $3.5 \times$ to $4.5 \times$ BP’s data passes and $3.6 \times$ to $4.4 \times$ its wall-clock time to reach its own best accuracy, which converts a per-pass advantage of 12.5 % to 19.9 % into an end-to-end cost of $3.2 \times$ to $4.0 \times$. The measurements argue against the alternatives: the cost of a pass favours MF; the cache that removes the recomputation is bit-exact and saves no time; and every ladder arm is launch-bound below 21 % utilisation (Section 5.5), which makes faster arithmetic an unlikely remedy, although it does not entirely exclude an implementation-maturity effect. What remains is the optimisation schedule: the number of passes a layer-sequential method needs before each stage converges, which is a tractable target.

Three statements bound the result. *Scope*: the 12.5 % to 19.9 % energy and 24.6 % to 27.5 % memory figures are measured on the 3×2000 MLPs; at 1.8 M weights the memory advantage has not yet appeared, and we ran no convolutional MF. Flattening CIFAR for the MLP protocol caps accuracy far below the 85.6 % our own 3-block CNN reaches under BP, so these comparisons are internally valid and say nothing about a competitive vision pipeline. *Accuracy under the budget*: equivalence to BP is established for CIFAR-100 under the matched data-pass protocol, and the budget costs MF 1.11 pp on CIFAR-10, so the matched data-pass comparison should be read as a cost result. *Hardware*: one accelerator, one driver. Board power at this utilisation is dominated by the static and clock-domain components of a large A100 die, so a smaller or better-saturated device would likely narrow the power gap; the memory saving and the pass multiplier, which follow from activation lifetime and from the optimisation, should carry across.

Implications. For a practitioner training 3×2000 MLPs under a fixed pass budget on this hardware, MF is the cheaper choice: lower on energy, power and memory, more so in the larger configurations, and on CIFAR-100 at no accuracy cost. For anyone training to convergence it reaches accuracy that BP does not, at $3.2 \times$ to $4.0 \times$ BP’s energy. The methodological point generalises: an efficiency claim about a training algorithm is uninterpretable without its stopping rule, which is a term in it, not a detail.

7 Conclusion

We measured the cost of forward-only training on hardware over almost 1000 runs. Given the same number of data passes as BP, Mono-Forward uses 12.5 % to 19.9 % less energy and 24.6 % to 27.5 % less per-process memory on the 3×2000 MLPs, at wall-clock time within 3 %, with accuracy statistically equivalent to BP’s on CIFAR-100; trained to its own convergence it exceeds BP’s test accuracy by 3.90 and 5.04 pp on the two CIFAR tasks. A pre-registered ablation ladder identifies gradient detachment, not the localised objective, as the dominant factor tested. The pass multiplier separates per-pass from end-to-end advantage.

Future work should first reduce the number of passes MF needs, for example through per-stage learning-rate schedules and adaptive per-stage stopping criteria. Convolutional and deeper architectures, training sets of varying size for a fixed task, and other accelerators would then show how far the result extends.

Importantly, the results matter most at scale: activation memory and energy are major costs of training large models, including large language models, and the saving we measure is larger in the larger configurations tested. Whether it persists at that scale is untested here, and establishing it is the natural next step.

Reproducibility

Code, configurations and the tidy table of 19944 measurements behind every number in this paper are available at <https://github.com/Przemyslaw11/BeyondBackpropagation>; the per-run JSON records and 1027 NVML power traces are archived and available from the authors. The released analysis scripts regenerate every number, table body and figure above.

Author Contributions. The research was conducted as part of a master’s thesis by Przemysław Spyra, under the supervision of Witold Dzwinel. Przemysław Spyra was responsible for the development and implementation of the methodology, experiments, and analysis. Witold Dzwinel provided conceptual guidance, critical feedback, and helped shape the direction and presentation of the work.

Acknowledgments. This work was supported by the National Science Centre (NCN), Poland, under grant OPUS-29, DEC-2025/57/B/ST6/04377, and by the Ministry of Science and Higher Education of Poland. The authors wish to thank the PLGrid Infrastructure for its support and ACK Cyfronet AGH for providing the computational resources on the Athena cluster under grant PLG/2025/018341, which were indispensable for this work. We gratefully acknowledge AGH University of Krakow for the institutional support and academic environment that made this research possible. We also thank the reviewers, whose questions about the deep supervision control and implementation maturity directed the two experiments that most changed this paper.

Disclosure of Interests. The authors have no competing interests to declare.

References

1. Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M.: Optuna: A next-generation hyperparameter optimization framework. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. pp. 2623–2631 (2019). <https://doi.org/10.1145/3292500.3330701>
2. Friday, B.B., Ali, D.A.: Balancing innovation and sustainability: Assessing the impact of generative AI on energy consumption. *International Journal of Innovative Research & Development* **13**(9), 1–10 (2024). <https://doi.org/10.24940/ijird/2024/v13/i9/sep24021>
3. Gong, J., Li, B., Abdulla, W.: Mono-Forward: Revisiting Forward-Forward through objective-locality decomposition. *arXiv preprint arXiv:2501.09238* (2025). <https://doi.org/10.48550/arXiv.2501.09238>
4. He, K., Zhang, X., Ren, S., Sun, J.: Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In: Proceedings of the IEEE International Conference on Computer Vision (ICCV). pp. 1026–1034 (2015). <https://doi.org/10.1109/ICCV.2015.123>

5. Hinton, G.: The Forward-Forward algorithm: Some preliminary investigations. arXiv preprint arXiv:2212.13345 (2022). <https://doi.org/10.48550/arXiv.2212.13345>
6. Ishikawa, S., Yokota, R., Karakida, R.: Local loss optimization in the infinite width: Stable parameterization of predictive coding networks and target propagation. In: Proceedings of the 13th International Conference on Learning Representations (ICLR) (2025). <https://doi.org/10.48550/arXiv.2411.02001>
7. Jaderberg, M., Czarnecki, W.M., Osindero, S., Vinyals, O., Graves, A., Silver, D., Kavukcuoglu, K.: Decoupled neural interfaces using synthetic gradients. In: Proceedings of the 34th International Conference on Machine Learning (ICML). Proceedings of Machine Learning Research, vol. 70, pp. 1627–1635 (2017). <https://doi.org/10.48550/arXiv.1608.05343>
8. Jouppi, N.P., Young, C., Patil, N., Patterson, D., Agrawal, G., Bajwa, R., Bates, S., Bhatia, S., Boden, N., Borchers, A., et al.: In-datacenter performance analysis of a tensor processing unit. In: Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA). pp. 1–12 (2017). <https://doi.org/10.1145/3079856.3080246>
9. Journé, A., Garcia Rodriguez, H., Guo, Q., Moraitis, T.: Hebbian deep learning without feedback. In: Proceedings of the 11th International Conference on Learning Representations (ICLR) (2023). <https://doi.org/10.48550/arXiv.2209.11883>
10. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. In: Proceedings of the 3rd International Conference on Learning Representations (ICLR) (2015). <https://doi.org/10.48550/arXiv.1412.6980>
11. Lakens, D.: Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science* **8**(4), 355–362 (2017). <https://doi.org/10.1177/1948550617697177>
12. Lee, C.Y., Xie, S., Gallagher, P., Zhang, Z., Tu, Z.: Deeply-supervised nets. In: Proceedings of the 18th International Conference on Artificial Intelligence and Statistics (AISTATS). Proceedings of Machine Learning Research, vol. 38, pp. 562–570 (2015). <https://doi.org/10.48550/arXiv.1409.5185>
13. Lillicrap, T.P., Santoro, A., Marris, L., Akerman, C.J., Hinton, G.: Backpropagation and the brain. *Nature Reviews Neuroscience* **21**(6), 335–346 (2020). <https://doi.org/10.1038/s41583-020-0277-3>
14. Lorberbom, G., Gat, I., Adi, Y., Schwing, A., Hazan, T.: Layer collaboration in the Forward-Forward algorithm. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 38, pp. 14141–14148 (2024). <https://doi.org/10.1609/aaai.v38i13.29324>
15. Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. In: Proceedings of the 7th International Conference on Learning Representations (ICLR) (2019). <https://doi.org/10.48550/arXiv.1711.05101>
16. Malladi, S., Gao, T., Nichani, E., Damian, A., Lee, J.D., Chen, D., Arora, S.: Fine-tuning language models with just forward passes. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 36, pp. 53038–53075 (2023). <https://doi.org/10.52202/075280-2308>
17. Papachristodoulou, A., Kyrkou, C., Timotheou, S., Theodoridis, T.: Convolutional channel-wise competitive learning for the Forward-Forward algorithm. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 38, pp. 14536–14544 (2024). <https://doi.org/10.1609/aaai.v38i13.29369>
18. Patterson, D., Gonzalez, J., Le, Q., Liang, C., Munguia, L.M., Rothchild, D., So, D.R., Texier, M., Dean, J.: Carbon emissions and large neural network training. arXiv preprint arXiv:2104.10350 (2021). <https://doi.org/10.48550/arXiv.2104.10350>

19. Pinchetti, L., Salvatori, T., Yordanov, Y., Millidge, B., Song, Y., Lukasiewicz, T.: Predictive coding beyond Gaussian distributions. In: *Advances in Neural Information Processing Systems (NeurIPS)*. vol. 35, pp. 1280–1293 (2022). <https://doi.org/10.52202/068431-0094>
20. Ren, M., Kornblith, S., Liao, R., Hinton, G.: Scaling forward gradient with local losses. In: *Proceedings of the 11th International Conference on Learning Representations (ICLR)* (2023). <https://doi.org/10.48550/arXiv.2210.03310>
21. Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. *Nature* **323**(6088), 533–536 (1986). <https://doi.org/10.1038/323533a0>
22. Salvatori, T., Mali, A., Buckley, C.L., Lukasiewicz, T., Rao, R.P.N., Friston, K., Ororbia, A.: Brain-inspired computational intelligence via predictive coding. *arXiv preprint arXiv:2308.07870* (2023). <https://doi.org/10.48550/arXiv.2308.07870>
23. Salvatori, T., Song, Y., Hong, Y., Sha, L., Frieder, S., Xu, Z., Bogacz, R., Lukasiewicz, T.: Associative memories via predictive coding. In: *Advances in Neural Information Processing Systems (NeurIPS)*. vol. 34, pp. 3874–3886 (2021). <https://doi.org/10.48550/arXiv.2109.08063>
24. Salvatori, T., Song, Y., Yordanov, Y., Millidge, B., Sha, L., Emde, C., Xu, Z., Bogacz, R., Lukasiewicz, T.: A stable, fast, and fully automatic learning algorithm for predictive coding networks. In: *Proceedings of the 12th International Conference on Learning Representations (ICLR)* (2024). <https://doi.org/10.48550/arXiv.2212.00720>
25. Schuirmann, D.J.: A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics* **15**(6), 657–680 (1987). <https://doi.org/10.1007/BF01068419>
26. Strubell, E., Ganesh, A., McCallum, A.: Energy and policy considerations for deep learning in NLP. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*. pp. 3645–3650. Association for Computational Linguistics (2019). <https://doi.org/10.18653/v1/P19-1355>
27. Tripp, C.E., Perr-Sauer, J., Gafur, J., Nag, A., Purkayastha, A., Zisman, S., Bensen, E.A.: Measuring the energy consumption and efficiency of deep neural networks: An empirical analysis and design recommendations. *arXiv preprint arXiv:2403.08151* (2024). <https://doi.org/10.48550/arXiv.2403.08151>
28. World Economic Forum, Accenture: Artificial intelligence’s energy paradox: Balancing challenges and opportunities. *AI Governance Alliance Report*, World Economic Forum (January 2025)
29. Zhao, G., Wang, T., Jin, Y., Lang, C., Li, Y., Ling, H.: The cascaded forward algorithm for neural network training. *Pattern Recognition* **161**, 111292 (2025). <https://doi.org/10.1016/j.patcog.2024.111292>